CloudInn PMS - Evidências de Arquitetura Backend v3.0.0

Alunos: Alyson Ferreira de Souza; Flavia Cristina Fagundes; Lucas Aioria Serpa; Matheus Pereira Siqueira

Escopo

BFF stateless, Hotel Service e Reservas/Guests Service foram estruturados em fatias verticais e Clean Architecture conforme o arc42. O BFF está preparado para Azure Container Apps.

Evidência no GitHub

https://github.com/LucasAioriaSerpa/CloudInn/tree/nova-arquitetura/backend

Regras arquiteturais

A suíte ArchiUnitTS nativa verifica isolamento de domínio/features, fatias verticais e ausência de dependência HTTP na aplicação.

Execução registrada

```
> cloudinn-root@0.1.0 test:backend
> node --test --test-reporter spec backend/test/*.test.js backend/*/test/*.test.js

' BFF agrega hotel, reservas e analytics em paralelo (1.683874ms)
' BFF marca resposta parcial quando analytics não é solicitado (0.281435ms)
' Hotel Service permite concluir limpeza e publica evento (1.043007ms)
' Hotel Service rejeita transição de status inválida (0.43355ms)
' Reservas Service cria reserva confirmada com idempotência (1.131713ms)
' Reservas Service responde conflito para sobreposição (0.617242ms)
' ArchiUnitTS: cada service mantém domínio e features sem dependência de infraestrutura (2.680505ms)
' ArchiUnitTS: casos de uso ficam em fatias verticais e não importam HTTP (0.800334ms)
!9 tests 8
!9 suites 0
!9 pass 8
!9 fail 0
!9 cancelled 0
!9 skipped 0
!9 todo 0
!9 duration_ms 241.612176
```

Trecho do teste arquitetural

```
const test = require('node:test');
const assert = require('node:assert/strict');
const fs = require('node:fs');
const path = require('node:path');

const backendRoot = path.resolve(__dirname, '..');
const services = ['bff', 'hotel-service', 'reservations-service'];
const sourceFiles = directory => fs.readdirSync(directory, { withFileTypes: true }).flatMap(entry => {
  const fullPath = path.join(directory, entry.name);
  if (entry.isDirectory()) return sourceFiles(fullPath);
  return fullPath.endsWith('.js') ? [fullPath] : [];
});

const importsOf = file => [...fs.readFileSync(file, 'utf8').matchAll(/require\(['"](\.\.?\/[^'"]+)['"]\)/
g)].map(match => match[1]);

test('ArchiUnitTS: cada service mantém domínio e features sem dependência de infraestrutura', () => {
  for (const service of services) {
    const src = path.join(backendRoot, service, 'src');
    for (const file of sourceFiles(src)) {
      const relative = path.relative(src, file).replaceAll(path.sep, '/');
      if (!relative.startsWith('domain/') && !relative.startsWith('features/')) continue;
      for (const imported of importsOf(file)) {
        assert.ok(!imported.includes('/infrastructure/'), `${service}/${relative} viola Clean
Architecture: ${imported}`);
      }
    }
  }
});

test('ArchiUnitTS: casos de uso ficam em fatias verticais e não importam HTTP', () => {
  for (const service of services) {
    const features = path.join(backendRoot, service, 'src', 'features');
    for (const file of sourceFiles(features)) {
      const relative = path.relative(features, file).replaceAll(path.sep, '/');
      assert.ok(relative.includes('/'), `${service}/${relative} deve estar dentro de uma fatia vertical`);
      assert.doesNotMatch(fs.readFileSync(file, 'utf8'), /express|http\.createServer|req\.|res\./,
`${service}/${relative} mistura transporte com aplicação`);
    }
  }
});
```